SELF-REFLECTIVE REPORT

BIT1123 Object Oriented Programming

Assignment 1 - Individual (20%)

Full Name	Eduardo Jorge Nhacule Junior
Student ID	202505010454
Class Code	BIT1123 OOP
Programme	Bachelor in Information Technology
NRIC / Passport No.	AB1897374
Course	BIT1123 / BISE2093 / DIT1113 - Object Oriented Programming
Assignment	Assignment 1 - Individual (20%)
GitHub Repository URL	https://github.com/eduardo-nhacule/BIT1123-OOP

1. Introduction

This report is a self-reflective account of my learning journey throughout the BIT1123 Object Oriented Programming course. Across ten weekly tutorials, I moved from writing my very first Java program to building a working graphical quiz application, progressively learning the core pillars of Object-Oriented Programming: encapsulation, inheritance, polymorphism, and abstraction. This report documents what I learned in each tutorial, the challenges I faced along the way, how I resolved them, the improvements I noticed in my own programming ability, and my plans for continuing to develop as a Java programmer beyond this course.

2. Knowledge Gained From Each Tutorial

Week 1: Introduction to Java & Basic Program Structure

The first tutorial introduced the fundamentals of Java as a programming language, starting with a simple HelloWorld.java program to understand the structure of a Java class, the main method, and how to compile and run a program. I then built on this with a StudentGrade.java program that used variables of different data types (String, int, char) together with if-else if-else conditional statements to convert numeric marks into a letter grade. This tutorial gave me my first real exposure to Java syntax, the print statements used for output, and the basic decision-making logic that almost every later program depends on.

Week 2: Classes, Objects and Constructors

This week moved from procedural-style code into actual object-oriented thinking. I created a Student class with attributes (name, age, gpa), a constructor to initialise those attributes when an object is created, and several methods (displayInfo, study, takeExam) that operate on the object's data. In Main.java I instantiated a Student object and called its methods. This was where the idea of a 'class as a blueprint' and an 'object as an instance' first became concrete for me, rather than just a textbook definition.

Week 3-4: Inheritance and Method Overriding

In this tutorial I designed a small class hierarchy with a base Person class containing shared attributes (name, id) and a general introduce() method. I then created Student and Lecturer subclasses that extend Person and override introduce() to provide their own specific behaviour. The Main.java driver class created objects of all three types and called introduce() on each, demonstrating how the same method call produces different output depending on the actual object type. This tutorial helped me understand inheritance as a way to reuse code and polymorphism as a way to let subclasses customise behaviour.

Week 5: Encapsulation, Getters and Setters

This tutorial focused on encapsulation. I built a Student class where all attributes (studentID, name, cgpa, programme) were declared private, and access to them was only possible through public getter and setter methods. As part of the tutorial documentation, I had to explain why fields are declared private (to restrict direct external access and enforce encapsulation), what would happen if they were

public (other classes could read or modify them directly and bypass any validation), and why getters and setters are used (they give controlled access to private fields, allow validation, and make the class easier to maintain without exposing its internal implementation). Writing out these justifications helped me understand encapsulation not just as a syntax rule but as a design principle.

Week 6: Protected Access and Multi-level Inheritance

Building on the earlier inheritance tutorial, this week introduced the protected access modifier. I created an Employee base class with protected id and name fields and a displayInfo() method, then extended it with a Lecturer subclass that added its own private fields (subject, department) and an additional displaySubject() method, while still reusing the constructor and behaviour of the parent class through super(). This clarified the difference between private, protected, and public access, and showed how protected fields can be accessed directly by subclasses while still being hidden from unrelated classes.

Week 7: Abstract Classes and Abstraction

This tutorial introduced abstract classes through an Appliance hierarchy. The abstract Appliance class defined shared fields and concrete methods (displayBrand, turnOn, turnOff) as well as an abstract operate() method with no implementation. Two subclasses, WashingMachine and Refrigerator, extended Appliance and each provided their own implementation of operate(). In Main.java, both subclasses were referenced through the parent Appliance type, and calling operate() on each produced different behaviour. This tutorial made the purpose of abstraction clear to me: defining a common contract in the parent class while forcing each subclass to implement the details relevant to itself.

Week 8-9: Collections and File Handling

This tutorial combined several concepts: using an ArrayList to store a dynamic list of tasks entered by the user through a Scanner, then writing that data to a text file using BufferedWriter/FileWriter, and finally reading the same data back using BufferedReader/FileReader. I also had to wrap the file operations in try-catch blocks to handle IOException. This was my first real experience with persistent data (saving information outside the running program) and with Java's exception-handling mechanism, both of which are essential for building programs that interact with real-world data.

Week 10: GUI Programming with Java Swing

The final tutorial introduced graphical user interface (GUI) programming using Java Swing. I created a Questions class to model a quiz question with two options and a correct answer, and a QuizBattleGUI class that extends JFrame and implements ActionListener to build an interactive quiz window with labels and buttons. Clicking a button triggers the actionPerformed() event handler, which checks the selected answer against the correct one and updates a result label accordingly. This tutorial tied together everything learned earlier in the semester (classes, objects, encapsulation) while introducing event-driven programming, which is a very different way of thinking compared to the top-to-bottom console programs from earlier weeks.

3. Challenges Encountered

Several challenges came up repeatedly while working through the tutorials:

- Understanding when to use private versus protected versus public access modifiers, and why encapsulation matters beyond simply 'hiding' variables.
- Getting comfortable with constructors, particularly using super() correctly to call a parent class constructor when building multi-level class hierarchies such as Person → Student/Lecturer and Employee → Lecturer.
- Debugging null pointer and compilation errors caused by mismatched method signatures when overriding methods such as introduce() and operate().
- Handling checked exceptions (IOException) correctly when reading from and writing to files in Week 8-9, since Java forces these to be caught or declared.
- Learning event-driven programming in Week 10, which required a completely different mental model from the top-to-bottom console programs used in earlier weeks — understanding how addActionListener() and actionPerformed() connect a GUI button click to my own logic took extra practice.

4. How the Challenges Were Overcome

I worked through most of these issues by re-reading the lecture material and Java documentation, and by deliberately tracing through my code line by line to understand the flow of execution rather than guessing at fixes. For the access-modifier confusion, I compared the three modifiers side by side using small test cases, which made the practical difference between private, protected, and public much clearer. For constructor and inheritance errors, I found it useful to draw out the class hierarchy on paper before coding, so I could see exactly which constructor needed to call which. For the file-handling exceptions, I paid closer attention to the try-catch structure and used the exception message itself (e.getMessage()) to diagnose what had gone wrong instead of treating errors as a black box. For the GUI tutorial, I experimented by changing small parts of the QuizBattleGUI code (such as adding print statements inside actionPerformed()) to observe exactly when each event fired, which helped the event-driven model click into place.

5. Improvements in Java Programming Skills

Comparing my Week 1 HelloWorld.java program to my Week 10 QuizBattleGUI.java, the improvement in my Java skills is clear. I am now comfortable defining classes with private fields and public accessor methods, building constructors, working with inheritance hierarchies and method overriding, using abstract classes to define shared contracts, working with core collections such as ArrayList, handling files and exceptions safely, and building simple interactive GUIs with Swing. My code is also noticeably better organised: I now separate responsibilities across classes (for example, keeping the Questions data model separate from the QuizBattleGUI presentation logic) instead of writing everything inside a single class as I did in the earliest tutorials.

6. Understanding of Object-Oriented Programming Concepts

- Encapsulation - keeping fields private and exposing controlled access through getters and setters, as practised in the Student class in Week 5.
- Inheritance - reusing and extending behaviour from a parent class, as seen in Person → Student/Lecturer (Week 3-4) and Employee → Lecturer (Week 6).
- Polymorphism - calling the same method (introduce(), operate()) on different object types and getting different behaviour depending on the actual class, as demonstrated in Week 3-4 and Week 7.
- Abstraction - defining a general contract in an abstract class (Appliance) and leaving the specific implementation to subclasses (WashingMachine, Refrigerator) in Week 7.

Together, these tutorials helped these four pillars stop being abstract definitions from a textbook and become practical tools I actively reach for when designing a program.

7. Future Learning Plans

- Deepen my understanding of interfaces and multiple inheritance of type, which were only briefly touched on this semester.
- Learn more advanced Java collections (HashMap, HashSet, TreeMap) and when to choose each over ArrayList.
- Explore exception handling in more depth, including creating and using custom exception classes.
- Build larger Swing or JavaFX applications to strengthen my GUI and event-driven programming skills beyond the simple quiz example.
- Practise applying design patterns (such as MVC) to structure larger Java projects more cleanly.
- Continue using GitHub for version control on future projects to build good software engineering habits early.

8. Conclusion

Overall, the ten tutorials in BIT1123 Object Oriented Programming took me from writing a single-line HelloWorld.java program to building a functioning GUI-based quiz application that applies encapsulation, inheritance, and abstraction together. The consistent, weekly hands-on practice was the single biggest factor in building my confidence with Java, and consolidating all of this work into one organised GitHub repository has also given me a first real taste of professional software development practices such as version control and documentation. I feel significantly more confident in my Java and OOP fundamentals than I did at the start of the semester, and I have a clear list of areas I intend to keep developing going forward.

9. GitHub Repository URL

All tutorial source code (Week 1 to Week 10), the README.md documentation, and this self-reflective report have been uploaded to the following GitHub repository:

https://github.com/eduardonhacule123-hub/Eduardo-Jorge-Nhacule_OBJECT-ORIENTED-PROGRAMMING.git